API

API API
API

LangChain embedding workflow ID API
BNY

1

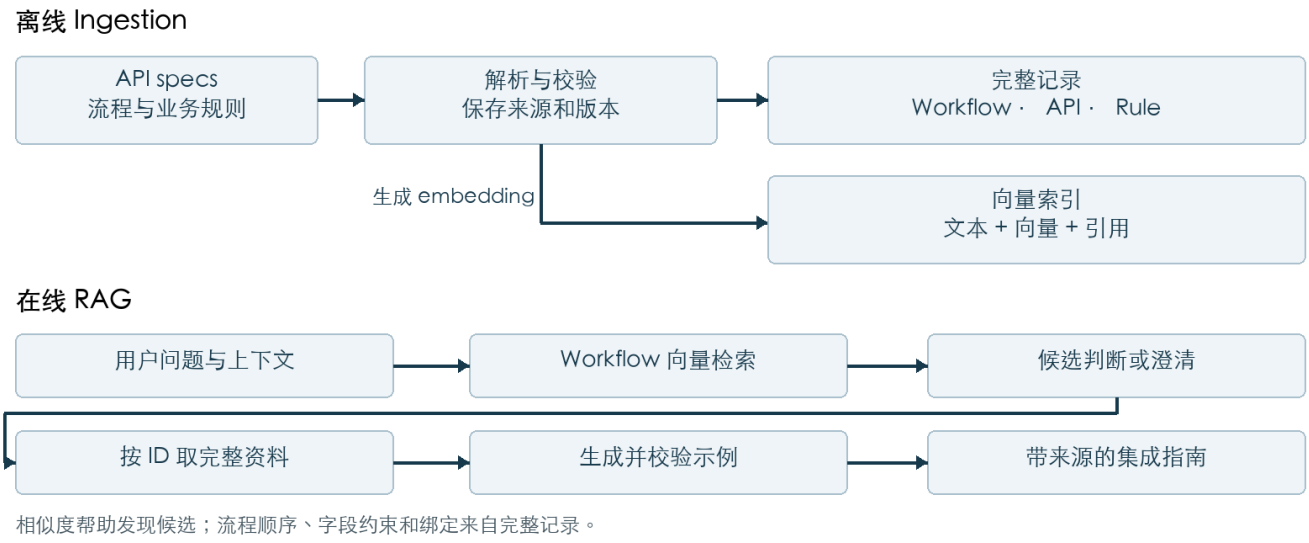
“ API ”
customerId verificationId fullName API

API mock server

Workflow scenario

Workflow Scenario “ ” “
” scenario workflow workflow

open_personal_account open_business_account workflow API
operation



Ingestion RAG LLM
API workflow registry

2

Embedding Vector store metadata API
JSON/YAML Chroma
LangChain Chroma

□□□□	□□□□□□□	□□□□□□□
	OpenAPI workflow	
Workflow	ID operation rule	
API operation	API spec operationId method/path schema	
Rule	ID	
Vector record	embedding	

workflow

embedding document_type API metadata ID

```
text: "                    "
metadata:
  document_type: workflow
  workflow_id: open_personal_account
  workflow_version: "1.0"
  bundle_id: demo-v1
  source_ref: workflows/personal.yaml
```

get_workflow("open_personal_account", "1.0", "demo-v1") workflow
operation_refs customer-api / v1 / createCustomer operation rule_refs LLM
schema

operationId API API operation API spec operationId
Workflow API spec bundle_id

3 Ingestion

OpenAPI workflow OpenAPI method path operationId
schema $ref schema OpenAPI
3.1.1

workflow LLM
workflow

step operation rule_refs
operationId schema JSON Pointer

embedding

□□□□	□□□□□□	□□□□□□□□□□
Workflow		ID
API	operation	API schema
Rule		ID

workflow API rule workflow API/rule

embedding metadata embedding embedding

active bundle bundle workflow API schema

4

workflow

1. “ ” “ ” “ ” “ ”

2. query embedding document_type=workflow bundle

3. top-k ID workflow k

4. LLM matched needs_clarification unsupported matched workflow ID needs_clarification LangChain

5. ID

“ ” “ ”

workflow operation workflow operation bundle top-k

API schema API schema

“ ” ID “ ”

schema

Schema API

□□□□	□□□□	□□□□□□
matched		
needs_clarification		
unsupported		
data_error		

5

operation

workflow 1.0 demo-v1 PASSED

□□	**Operation □□□**	□□□□□□□
1	customer-api / v1 / createCustomer POST /customers	customerId
2	identity-api / v1 / verifyIdentity POST /identity-verifications	customerId verificationType verificationId status
3	account-api / v1 / createAccount POST /accounts	customerId verificationId currency accountId

□□	**open_personal_account**	**open_business_account**
customerType	PERSONAL	BUSINESS
	fullName	legalName companyRegistrationNumber
verificationType	PERSON_ID	COMPANY_REGISTRATION
currency	USD EUR	USD EUR

Dummy OpenAPI schema oneOf API

“ API ” USD
open_personal_account operation workflow

```
workflow_id: open_personal_account
workflow_version: "1.0"
bundle_id: demo-v1
steps:
  - id: customer
    operation_ref: customer-api/v1/createCustomer
  - id: verification
    operation_ref: identity-api/v1/verifyIdentity
    bindings:
      customerId: customer.response.customerId
  - id: account
    operation_ref: account-api/v1/createAccount
    requires: verification.response.status == PASSED
    bindings:
      customerId: customer.response.customerId
      verificationId: verification.response.verificationId
rule_refs:
  - personal-customer/1.0
  - verification-gate/1.0
```

POST /customers PERSONAL Alex Example

```
{"customerType": "PERSONAL", "fullName": "Alex Example"}
```

mock {"customerId": "cus_demo_001"} POST /identity-verifications ID PERSON_ID

```
{"customerId": "cus_demo_001", "verificationType": "PERSON_ID"}
```

mock {"verificationId": "ver_demo_001", "status": "PASSED"} POST /accounts ID USD dummy schema

```
{
  "customerId": "cus_demo_001",
  "verificationId": "ver_demo_001",
  "currency": "USD"
}
```

body spec ID ID FAILED

“ 1.0 createCustomer → verifyIdentity → createAccount customerId PERSONAL customerId verificationId ID ” workflows/personal.yaml customer API request schema verification-gate 1.0

6 LangChain

□□	□□□□	□□
	JSON/YAML	workflow operation rule
Embedding	LangChain Chroma	workflow metadata
	Python	ID

□□	□□□□	□□
	Schema	
Chat UI	Streamlit	

LangChain Streamlit chat_input chat_message
Streamlit Chat

```
bundle = pin_active_bundle()
context = prepare_query(message, conversation)
candidates = search_workflows(context, bundle)
decision = select_or_clarify(context, candidates)
validate_selection(decision, candidates)
if decision.status != matched: return decision
workflow = get_workflow(decision.id, decision.version, bundle)
evidence = load_operations_and_all_applicable_rules(workflow)
plan = resolve_steps_fields_and_bindings(workflow, evidence)
draft = generate_guide(context, plan, evidence)
return validate_examples_and_attach_sources(draft, evidence)
```

tool MCP

search_workflows get_workflow get_api_spec ID LLM
MCP workflow LangChain Tools MCP

LLM tools MCP server
prompt

ID operation/rule

□□□□□□□	□□□□□□
	workflow
	unsupported
LLM ID	
operation rule	data_error
ID	
PASSED	

workflow workflow

7

dummy OpenAPI workflow ingestion embedding

workflow registry

dummy spec
embedding mock
BNY

dummy

- OpenAPI 3.1.1 operation schema
- LangChain Chroma integration
- LangChain Structured output
- LangChain Tools
- MCP Architecture
- Streamlit Chat elements